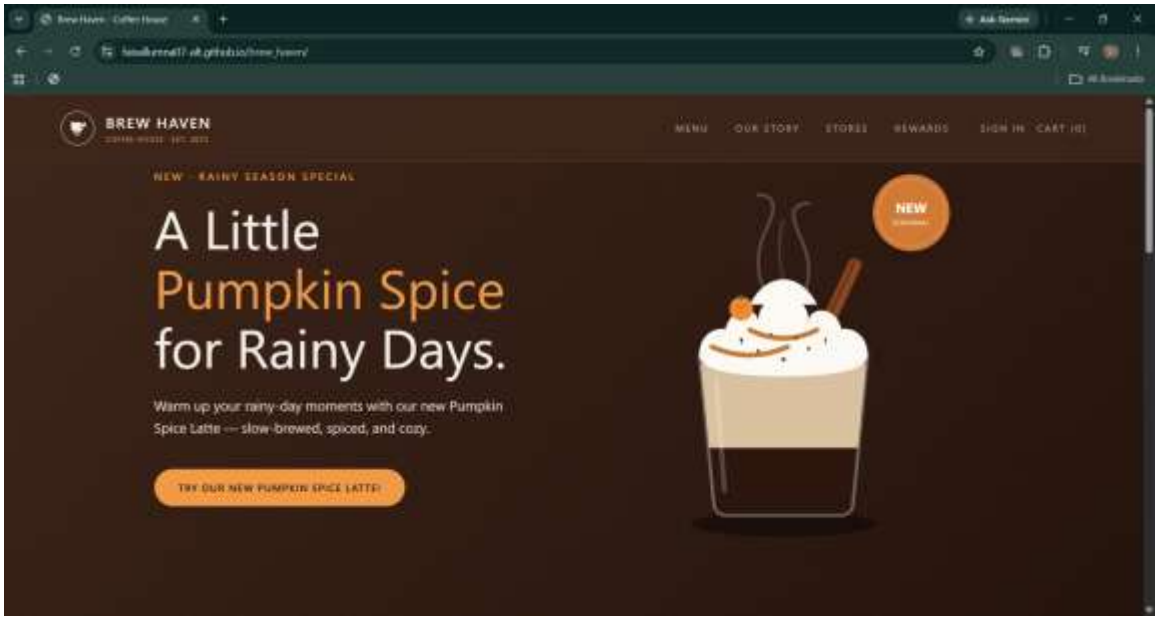
SOFTWARE QUALITY ASSURANCE

100-POINT ANTI-AI AUTHENTIC ASSESSMENT

"Review Your Own Software: An Evidence-Based SQA Inspection"

Name:	Mohammad, Faisal L.	**Section:**	III-CS3
Course/Subject:	Software Quality Assurance	**Date:**	Sept 19, 2026
Software/Project:	BREW HAVEN CAFÉ WEBSITE	**Role:**	Programmer

PART I — SELECT YOUR OWN SOFTWARE ARTIFACT [10 POINTS]
Screenshot/photo of the artifact:



Project title:
Brew Have Café Website

Your role in creating it:
Front end programmer

Date/year created:
September 1, 2026

Technology/language used:
HTML5, CSS3, Vanilla JavaScript

Short explanation of what the software does:
A single-page web application for a local coffee shop. It features a responsive landing page, a seasonal menu, dynamic cart functionality, and interactive user modals (login/sign in and cart).

Personal Statement:
I selected this artifact because it represents a complex UI/UX implementation with interactive

JavaScript components (login/sign in and cart) that are highly susceptible to logic, usability, and responsiveness defects.

PART II — DEFINE YOUR REVIEW PLAN [15 POINTS]

Objective:

To determine whether the Brew Haven Café website's authentication interface, cart and checkout logic, and responsive navigation behave correctly and safely under realistic user conditions specifically, whether the sign-in form actually validates credentials, whether cart data survives normal browsing behavior such as a page refresh, whether pricing data is consistent between what the customer sees and what the system computes, and whether every interactive control (Sign In, Cart, Checkout) remains reachable on screens 992px wide and below.

Scope:

- The <script> block in its entirety: openModal(), closeModal(), window.onclick, handleLogin(), addToCart(), removeFromCart(), updateCartUI(), checkout()
- The #loginModal and #cartModal markup
- The <nav> block and the .nav-actions container
- The .menu-grid markup, including the price displayed in .menu-price and the price passed to addToCart()
- The @media (max-width: 992px) rule
- The footer#stores block and all anchor targets

Out of scope:

- Visual/aesthetic judgments (colour palette, typography choices)
- The hand-drawn SVG illustrations, except where they affect accessibility
- Cross-browser rendering on browsers I do not have installed (noted as observations only, not as confirmed defects)
- Server-side concerns there is no backend to review

Artifact/Version:

Single file index.html, Brew Haven Café Website, version as of September 1, 2026. No version control history is available, which is itself a process weakness I discuss in Part VII.

Applicable criteria:

	Criterion	**Source**
C1	Authentication must verify submitted credentials before reporting success	Basic security requirement
C2	User-entered state must not be silently lost during normal use	Usability / data integrity

C3	Displayed data and computed data must come from one source of truth	Maintainability
C4	Currency and locale must match the stated business location	Business-rule correctness
C5	All functionality available on desktop must remain reachable on smaller screens	All functionality available on desktop must remain reachable on smaller screens
C6	Interactive controls must be reachable by keyboard	WCAG 2.1 – 2.1.1 Keyboard
C7	A link's label must match what the link actually does	Consistency / user trust
C8	System feedback must reflect what the system actually did	Honest feedback principle

Participants or reviewers:

Since this is an individual activity, I performed all roles myself: Mohammad, Faisal L.

Review method:

A static individual inspection supported by a self-walkthrough of three user scenarios.

Entry criteria:

1. The complete source file is available and readable.
2. The artifact loads in a browser with no console errors blocking execution.
3. The review checklist (C1–C8) is prepared before reading begins.
4. A defect log template is ready.

Exit criteria:

1. Every item in the declared scope has been read at least once against the checklist.
2. At least five findings are recorded with reproducible evidence and exact locations.
3. Every finding carries a classification, impact, severity, and priority with a written justification.
4. At least two findings have been reworked, and the rework has passed a follow-up re-review.

5. Any finding that is not corrected is formally deferred with a documented reason and an accepted risk, rather than quietly ignored.
6. A regression check confirms that the corrections did not break previously working behaviour.

PART III — PERFORM AN ACTUAL INSPECTION [25 POINTS]

The inspection followed the four categories in the activity. Below is what I read and what I found, including a few things I checked and cleared.

A. Requirements

The project has no written requirements document, so I inspected the site's own implied requirements what the interface promises the user.

The navigation bar advertises four destinations: MENU, OUR STORY, STORES, and REWARDS. Reading the markup, STORES and REWARDS point to the identical anchor. The footer does contain a small "Rewards Program" block, but a navigation item is a promise of a destination, and two menu labels resolving to one place is an omission — the rewards requirement was never really implemented, only gestured at.

A second requirement is a contradiction between business context and implementation. The footer states the store is at "Brgy. 3, Calamba City, Laguna, 4027," yet every price is expressed in US dollars ($5.50, $4.75, $4.50) and the total is formatted as $${totalPrice.toFixed(2)}. A Philippine café pricing a latte at $5.50 is not a styling preference; it is an incorrect business rule.

Third, the Sign In feature is unverifiable as written. There is no stated rule for what a valid credential is, no registration flow that could create one ("Create one" is href="#"), and no-account state after login. I cannot write a pass/fail test for a requirement that does not define success.

B. Design

The biggest design finding is the responsive strategy. The single media query contains:

```
563     @media (max-width: 992px) {
564         #home, #story .story-content {
565             flex-direction: column;
566             text-align: center;
567         }
568         .actions {
569             flex-direction: column;
570             justify-content: center;
571         }
572         .description { margin: 0 auto 30px auto; }
573         .content-right { margin-top: 60px; }
574         nav { display: none; /* Can be replaced with a hamburger menu later */ }
575         header { justify-content: center; }
576     }
```

The <nav> element does not only hold section links .nav-actions is nested inside it, and .nav-actions holds SIGN IN and CART. Hiding nav therefore hides the entire commerce entry point. The comment in my own code ("Can be replaced with a hamburger menu later") is the clearest possible evidence that this was a deliberate deferral that I then forgot to come back to.

On architecture: all state lives in a single page-scoped array, let cart = [], with no persistence layer at all. There is no localStorage, no sessionStorage, no server. The design has no concept of a session, so the cart cannot outlive the page.

On error handling: there is essentially none. handleLogin() has no failure branch, checkout() has no failure branch, and no function has a try/catch or a validation guard beyond the browser's native required attribute.

On coupling, window.onclick is assigned directly rather than registered with addEventListener. This works today because nothing else claims that handler, but it means any future global click listener will silently overwrite modal dismissal. I recorded this as an observation rather than a defect, because in the current artifact it does not misbehave.

C. Implementation

Reading the <script> line by line produced the most serious finding in this report.

```
796     function handleLogin(e) {
797         e.preventDefault();
798         alert('Sign in successful! Welcome to Brew Haven.');
799         closeModal('loginModal');
800         // Clear form
801         e.target.reset();
802     }
```

The function never reads #email or #password. The values the user types are collected by the form, ignored by the code, and then wiped by reset(). Any syntactically valid email plus any non-empty password produces the message "Sign in successful!" This is not an incomplete feature it is a feature that actively reports a false result.

checkout() behaves the same way:

```
871     function checkout() {
872         if (cart.length === 0) {
873             alert("Your cart is empty! Add some coffee first.");
874         } else {
875             alert("Thank you for your order! Your coffee will be ready soon.");
876             cart = []; // Empty cart
877             updateCartUI();
878             closeModal('cartModal');
879         }
880     }
```

The empty-cart guard is correct and I credit it. But the success branch destroys the cart, tells the customer the order is confirmed, and produces no order record, no reference number, and no request to anything. The destruction is irreversible.

On pricing, each product's price exists in two independent places:

```
667                 <div>
668                     <h3>Pumpkin Spice Latte</h3>
669                     <p>Our signature espresso blended with autumn spices, steamed milk, and topped with whipped cream.</p>
670                     <span class="menu-price">$5.50</span>
671                 </div>
672                 <button class="btn btn-outline" onclick="addToCart('Pumpkin Spice Latte', 5.50)">Add to Cart</button>
673             </div>
674             <div class="menu-item">
675                 <div>
676                     <h3>Maple Pecan Cold Brew</h3>
677                     <p>Slow-steeped cold brew kissed with sweet maple syrup and topped with crushed toasted pecans.</p>
678                     <span class="menu-price">$4.75</span>
679                 </div>
680                 <button class="btn btn-outline" onclick="addToCart('Maple Pecan Cold Brew', 4.75)">Add to Cart</button>
681             </div>
682             <div class="menu-item">
683                 <div>
684                     <h3>Cinnamon Chai Steam</h3>
685                     <p>A comforting blend of black tea, cinnamon, clove, and steamed oat milk for chilly afternoons.</p>
686                     <span class="menu-price">$4.50</span>
687                 </div>
688                 <button class="btn btn-outline" onclick="addToCart('Cinnamon Chai Steam', 4.50)">Add to Cart</button>
689             </div>
```

Nothing enforces that these two numbers agree. If I run a promo and edit only the visible span, the cart silently charges the old price.

I also checked money arithmetic. totalPrice += item.price * item.quantity uses binary floating-point, and.toFixed(2) only rounds at display time. I tested the specific combinations available in this menu and did not reproduce a visible wrong total, so I am not logging this as a confirmed defect I am recording it as a latent risk that I addressed opportunistically during rework.

I inspected removeFromCart(index) carefully because index-based removal from a re-rendered list is a classic stale-reference bug. It turns out to be correct here: updateCartUI() clears cartItemsContainer.innerHTML and rebuilds every row with a freshly computed index on each call, so no stale index can survive. I am noting this explicitly because part of an honest inspection is reporting what you cleared up, not only what you condemned.

One genuine gap in the cart is that quantity can only increase. addToCart() increments existingItem.quantity, but there is no decrement function the only reversal available is removeFromCart(), which deletes the whole line. A customer who accidentally clicks "Add to Cart" four times must delete all four and start over.

Finally, updateCartUI() injects ${item.name} into the DOM via innerHTML inside a template literal. Every name currently comes from a hardcoded literal in the HTML, so there is no exploitable path today. I am flagging this as a hardening item, not as an active vulnerability, because overstating it would be exactly the kind of exaggeration the activity warns against.

D. Documentation

The artifact has no README, no setup notes, and no inline documentation beyond section comments. Two of those comments are now misleading. /* Brief visual feedback */ sits directly above alert(...), but a native alert() is blocking and modal the opposite of brief and visual. And /* Can be replaced with a hamburger menu later */ documents an intention that was never fulfilled, which makes the comment a small piece of outdated documentation embedded in shipped code.

There is also dead CSS: the rules.link and link:hover is defined but no element in the document carries class="link". It is harmless at runtime, so I logged it as a maintainability observation rather than a defect.

Compatibility item not confirmed: backdrop-filter is used on both header and modal-overlay without a -webkit- prefix. Safari historically required the prefix. I could not test Safari on my machine, so under my own exit criteria I am reporting this as an unverified observation rather than a finding.

PART IV — EVIDENCE-BASED DEFECT LOG [20 POINTS]

Finding	Evidence / Location	Classification	Impact	Severity	Priority	Recommended Action
1. The Sign In form reports success for any input; no credential is ever checked.	<script> → handleLogin(e). The body is e.preventDefault(); alert('Sign in successful! …'); closeModal('loginModal'); e.target.reset(); — the values of #email and #password are never read.	Implementation / Security defect (logic omission)	The site makes a security claim it cannot support. A user believes an account exists and is protected when nothing was verified; a reviewer could reasonably assume auth is implemented.	Critical	High	Either implement real server-side authentication, or remove the false success message and relabel the modal as a non-functional demo until a backend exists. Do not ship the current wording.

2. All Sign In and Cart controls disappear on screens ≤ 992px, with no replacement.	@media (max-width: 992px) { nav { display: none; } }. .nav-actions (SIGN IN, CART, #cartCount) is a child of <nav>, so it is hidden too. The code comment reads /* Can be replaced with a hamburger menu later */.	Design defect (responsive/usability)	Mobile and tablet users can still press "Add to Cart," but can never open the cart or check out. For a café site, that is most of the real audience being blocked from the one transactional feature.	Major	High	Move .nav-actions outside <nav> or add a hamburger toggle so Sign In and Cart remain reachable at every breakpoint.
3. Cart contents are lost on refresh or navigation away.	<script> → let cart = []; declared in page scope. No localStorage, sessionStorage, cookie, or server write appears anywhere in the file.	Design defect (state persistence)	A customer who refreshes, follows a link, or reopens the tab silently loses their whole order and must rebuild it. Silent loss of user-entered data is a common abandonment trigger.	Major	Medium	Persist the cart to localStorage on every mutation and restore it on page load.
4. Each price is stored	.menu-grid → e.g. <span class="menu-	Implementation defect	Editing one	Major	High	Define prices once in a

twice, and the currency contradicts the stated store location.	price">$5.50 and onclick="addToCart('Pumpkin Spice Latte', 5.50)" on the same card (repeated for 4.75 and 4.50). Currency $ vs. footer address "Brgy. 3, Calamba City, Laguna, 4027."	(duplicated data) + Requirements defect (incorrect business rule)	price and not the other makes the cart charge a different amount than the menu advertises a billing discrepancy. Separately, USD pricing is simply wrong for a Calamba store.			single data source, render both the label and the cart total from it, and format using PHP (₱) via Intl.NumberFormat('en-PH', …).
5. Checkout declares an order confirmed and erases the cart, with no order captured anywhere.	<script> → checkout(): alert("Thank you for your order! …"); cart = []; updateCartUI(); closeModal('cartModal'); No fetch, no order ID, no storage, no payment step.	Implementation / Requirements defect (false positive feedback)	The customer is told a real order was placed. Nothing was placed, and their cart is gone with no undo. This is the most damaging possible combination: false confirmation plus irreversible data loss.	Major	High	Until an order backend exists, replace the confirmation with an explicit preview/demo message and stop clearing the cart. Once a backend exists, only clear after a successful response and show an order reference.
6. Three navigation/labelling mismatches: REWARDS leads to	<nav>: STORES and REWARDS share one target. .auth-links: <a	Requirements / Consistency defect	Users are sent somewhere other than what the	Moderate	Medium	Give Rewards its own section or remove the item; build a registration form or remove

STORES, "Create one" is a dead link, and "Sign up today" opens the Sign In modal.	href="#">Create one. Footer: Sign up today — opens sign-in, and href="#" is not prevented, so the page also jumps to the top.		label promised, which erodes trust in the rest of the navigation. The footer link additionally scrolls the page behind the open modal, which looks like a bug even to a casual visitor.			the two links; add event.preventDefault() to the footer link.
7. Key interactive controls cannot be reached or operated by keyboard.	<nav>: SIGN IN and the CART link have **no href**, so they are not in the tab order. .close-modal is a , not a button. No keydown/Escape handler exists. Note that nav a:hover, nav a:focus { color: var(--accent-color); } defines a focus style that can never trigger.	Design defect (accessibility, WCAG 2.1.1 Keyboard)	Keyboard-only and screen-reader users cannot open Sign In or Cart at all, and a user who somehow opens a modal cannot close it without a mouse. My own CSS proves I intended focus support and never verified it.	Moderate	Medium	Convert these controls to <button> elements, add role="dialog" and aria-modal="true" to the overlays, and handle Escape to close.

8. The STORES/REWARDS anchor scrolls the footer heading underneath the fixed header.	header { position: fixed; height: 90px; } versus footer { padding: 60px 0 30px; }. section compensates with padding-top: 100px, but footer#stores does not 60px of top padding is less than the 90px header, so roughly the top 30px of footer content is covered on arrival.	Implementation defect (layout/CSS)	The "Location" heading is partly obscured exactly when the user clicks the link that was supposed to show it. Recoverable by scrolling, so impact is limited.	Minor	Low	Increase footer top padding to at least 110px, or add scroll-margin-top: 100px to #stores.

PART V — THE “WHAT IF?” CHALLENGE [10 POINTS]

I selected Findings 1, 2, and 4 because they represent three different kinds of risk: a trust/security risk, an availability risk, and a financial-accuracy risk.

Finding 1 (Sign In accepts anything)

1. What could happen in production?

A visitor types their real email and a password they reuse on other accounts. The site answers "Sign in successful! Welcome to Brew Haven." From that point the user believes Brew Haven holds an account for them. They may then treat the site as a place where their identity is established for example, expecting their rewards points to be tracked, or later reporting a "hacked account" to the café when nothing of the sort exists. Worse, a user testing the site with a deliberately wrong password will also be let in, and if they understand what that implies, they will conclude the site's security is broken and stop trusting the ordering feature too.

2. Why could it happen?

Because I built the modal as a visual milestone. I wanted the layout, the blur overlay, and the form styling to look right, so I wrote the smallest handler that would let me close the modal and keep testing the CSS. That placeholder alert() was meant to be temporary, but nothing in my process ever flagged it again there is no TODO on it, no test that would fail, and the feature looks complete from the outside, which is precisely why it survived.

3. Who or what could be affected?

The customer most of all, since they may enter a reused password into a form that does nothing but discard it. Also, the café's reputation a security claim that turns out to be theatre is worse than no login button at all. And myself, professionally: if I submitted this to an employer as portfolio work, it would read as either careless or dishonest.

4. What evidence supports my prediction?

The function body itself. handleLogin(e) contains four statements and not one of them references document.getElementById('email') or #password. There is no comparison, no request, no conditional. The success message is unconditional, so by construction there is no input that produces a failure.

5. What should be changed?

Long term, authentication must move to a server that hashes and verifies credentials that cannot be fixed in front-end code, and pretending otherwise would be a second false claim. Short term, and this is what I can do now, the misleading message must go. The honest interim behavior is to state plainly that accounts are not yet available, so the interface stops asserting something that is not true.

Finding 2 (Navigation hidden below 992px)

1. What could happen in production?

A student sitting in the café opens the site on her phone to order. The menu renders beautifully the responsive stack works well. She taps "Add to Cart" on a Pumpkin Spice Latte and gets the confirmation alert, so she believes the order is in progress. Then she looks for the cart, and there is no cart. No header links at all, just a centered logo. She has an item in a cart she cannot open, cannot review, and cannot check out. She will assume the site is broken and close it.

2. Why could it happen?

I developed the whole site on a laptop at full width. The media query was written in one pass to stop the horizontal nav from colliding with the logo on narrow screens, and display: none was the fastest fix. At the time I was thinking of <nav> as "the section links," and I forgot that I had nested .nav-actions inside it for alignment purposes. One structural decision I made for CSS convenience quietly determined which users get access to the cart.

3. Who or what could be affected?

Realistically, most of the audience. A café website is something people open on a phone, often while nearby or on the way. Tablet users are affected as well, since the breakpoint is 992px, which catches most tablets in portrait and some small laptop windows. The business impact is that the one revenue-relevant feature is unavailable to the largest user segment.

4. What evidence supports my prediction?

The DOM nesting and the media query together. .nav-actions, containing both SIGN IN and CART (0), is a child of <nav>; nav {display: none;} applies at max-width: 992px. Meanwhile the Add-to-Cart buttons live inside menu-item, which is never hidden so addToCart() still runs and still fires its alert. The mismatch is not speculative; it is the direct consequence of the two rules. My own comment, /* Can be replaced with a hamburger menu later */, confirms I knew the fix was incomplete.

5. What should be changed?

.nav-actions must not be a descendant of the element being hidden, or the hidden element must be replaced by a real toggle. I implemented a hamburger toggle in Part VI so that every control available at 1400px is also available at 375px.

Finding 4 (Duplicate prices, wrong currency)

1. What could happen in production?

The café runs a rainy-season promo and drops the Cinnamon Chai Steam from 4.50 to 3.95. I edit the visible label because that is the number I can see on screen, and I miss the 4.50 sitting inside the onclick attribute. The menu now advertises one price while the cart quietly charges another. A customer notices the total does not match the menu and, quite reasonably, concludes the café is overcharging. Separately, the currency problem means a Calamba customer reads "$5.50" and mentally converts it to well over ₱300 for a latte, which is enough to make them leave before ordering anything.

2. Why could it happen?

Because I had no single source of truth. I wrote the card markup first to get the layout looking right, then wired the button afterwards and typed the number again from memory. The dollar sign came from the tutorials and design references I was copying from, and I never stopped to check it against the address I had already typed into my own footer. Both mistakes come from the same habit: treating the page as a picture to assemble rather than as data being presented.

3. Who or what could be affected?

The customer is affected financially, and the café is affected legally and reputationally a displayed price that differs from the charged price is a consumer-protection problem, not merely a bug. Future maintainers, including me next semester, are affected too, because the defect is invisible until someone changes a price and a discrepancy appears with no obvious cause.

4. What evidence supports my prediction?

Every one of the three menu cards contains the same literal number twice, in two syntactically unrelated places $5.50 and addToCart('Pumpkin Spice Latte', 5.50). There is no mechanism anywhere in the file that links them, so nothing can detect a mismatch. And cartTotalValue.innerText = \$${totalPrice.toFixed(2)}`` hardcodes the dollar sign a fourth time, directly contradicting the footer address.

5. What should be changed?

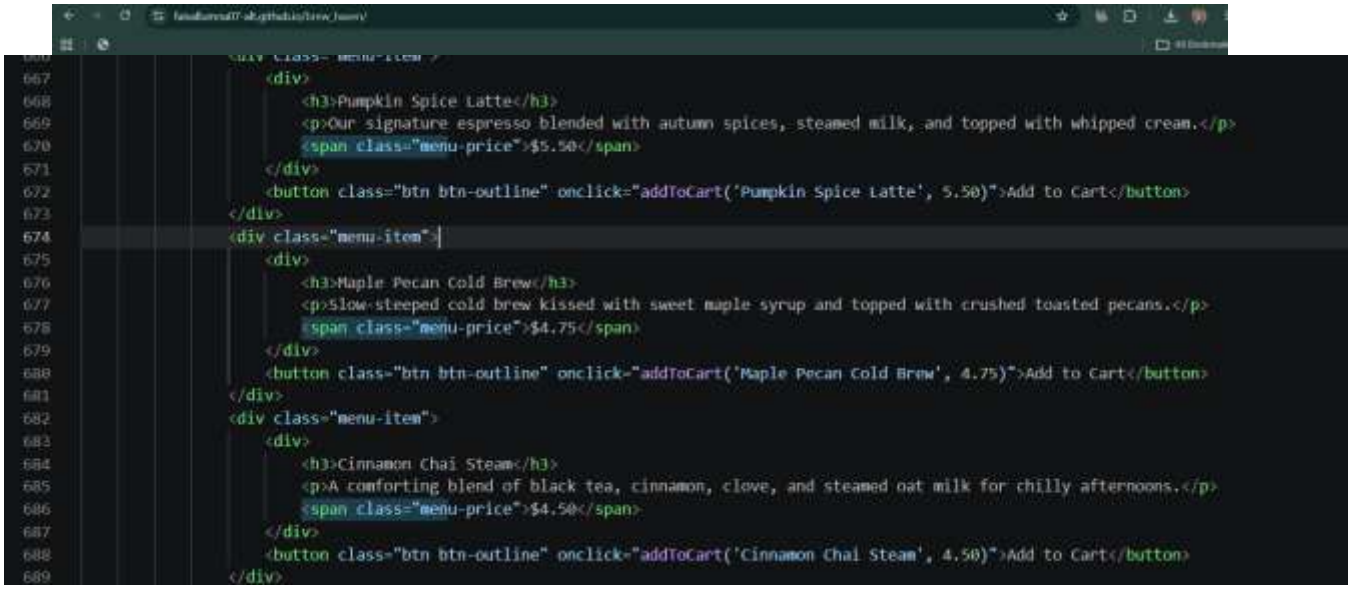
Prices must be declared once and read everywhere else. I moved them into data-* attributes with a single formatting function, switched to Philippine peso formatting, and stored amounts in centavos so the totals are computed with integers instead of floats. This is Rework #1 in Part VI.

PART VI — REWORK AND FOLLOW-UP [10 POINTS]

I corrected Findings 4 and 2.

Rework 1:

Before correction:



Code Evidence (original):

```
867             // Update total price
868             cartTotalValue.innerText = `$${totalPrice.toFixed(2)}`;
869         }
```

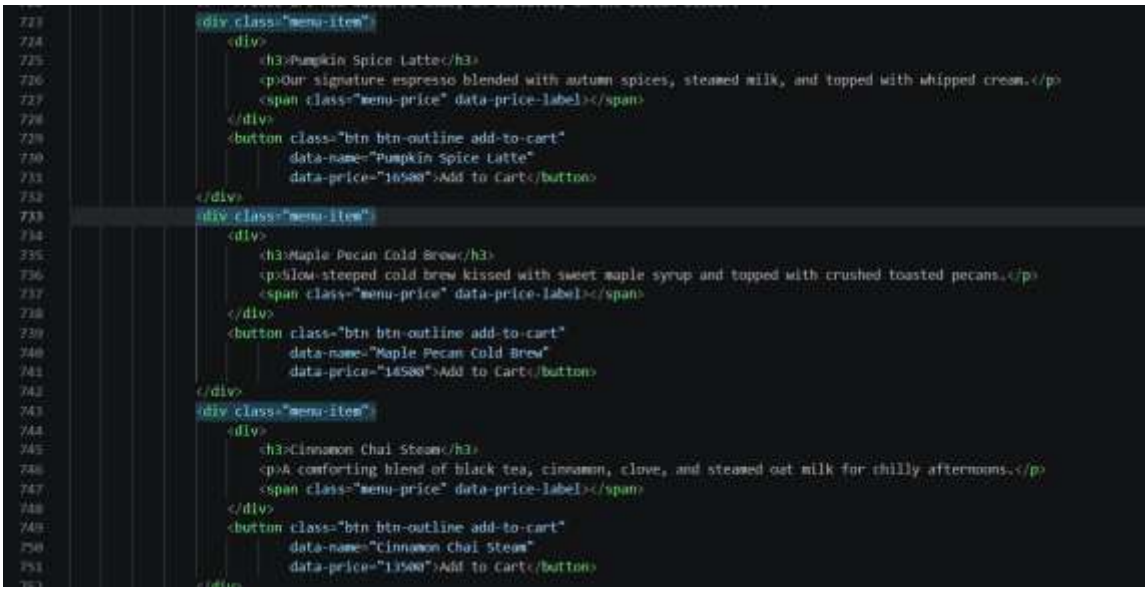
Explanation of the problem:

The price 5.50 is written twice per product with nothing connecting the two copies, so they can drift apart the moment anyone edits one. The $ symbol is hardcoded in four separate places and contradicts the store address in the footer. Totals are also accumulated as floating-point decimals, which is not the right representation for money.

After Correction:



Updated Code:



```
// --- Money helpers: store centavos as integers, format once. ---
const peso = new Intl.NumberFormat('en-PH', { style: 'currency', currency: 'PHP' });
function formatPrice(centavos) {
    return peso.format(centavos / 100);
}

// Render every visible price label from the same data-price the cart uses.
document.querySelectorAll('.add-to-cart').forEach(btn => {
    const label = btn.closest('.menu-item').querySelector('[data-price-label]');
    if (label) label.textContent = formatPrice(Number(btn.dataset.price));

    btn.addEventListener('click', () => {
        addToCart(btn.dataset.name, Number(btn.dataset.price));
    });
});

function addToCart(itemName, priceInCentavos) {
    // Check if item already exists in cart
    const existingItem = cart.find(item => item.name === itemName);

    if (existingItem) {
        existingItem.quantity += 1;
    } else {
        cart.push({ name: itemName, price: priceInCentavos, quantity: 1 });
    }

    updateCartUI();
```

Inside updatredCartUI():

```
cart.forEach((item, index) => {
    const itemTotal = item.price * item.quantity;
    totalPrice += itemTotal;

    const itemEl = document.createElement('div');
    itemEl.className = 'cart-item';
    itemEl.innerHTML = `
        <div class="cart-item-info">
            <h4>${item.name}</h4>
            <p>${formatPrice(item.price)} x ${item.quantity}</p>
        </div>
        <div>
            <div style="font-weight: bold; margin-bottom: 5px;">${formatPrice(itemTotal)}</div>
            <span class="cart-item-remove" onclick="removeFromCart(${index})">Remove</span>
        </div>
```

Explanation of what changed:

The price now exists in exactly one location per product the data-price attribute and both the visible label and the cart read from it, so a drift between the two is now structurally impossible rather than merely unlikely. Prices are stored as integer centavos (16500 instead of 165.00) so all arithmetic is integer arithmetic, which removes the floating-point risk I noted in Part III. Currency formatting happens in one function, formatPrice(), using Intl.NumberFormat('en-PH'), so the ₱ symbol is no longer typed by hand anywhere. I also replaced the inline onclick attributes with addEventListener, which separates behavior from markup and made the data-driven approach possible.

Follow up of Rework 1:

How I know the original problem is solved:

I searched for the literal strings 5.50, 4.75, 4.50, and for the $ character in a price context. After the fix, each price value appears exactly once, and no price-related $ remains. That is a verifiable property, not an impression. I then simulated the promo scenario that caused the defect in the first place: I changed one data-price from 13500 to 11900 and confirmed that both the menu label and the cart line item changed together, because they now come from the same attribute.

How I know I did not break something else (regression check):

Behaviour re-checked	Result
Add one item, cart count increments to 1	Pass
Add the same item twice, one line shows x 2 with double the line total	Pass
Add all three items, header count shows 3	Pass
Remove the middle item; remaining two keep correct names and totals	Pass
Empty the cart; "Your cart is currently empty." reappears and total resets to ₱0.00	Pass
Checkout on an empty cart still warns instead of confirming	Pass

Rework 2:



Code Evidence (original):

```
593         <nav>
594             <a href="#menu">MENU</a> <span class="divider">&middot;</span>
595             <a href="#story">OUR STORY</a> <span class="divider">&middot;</span>
596             <a href="#stores">STORES</a> <span class="divider">&middot;</span>
597             <a href="#stores">REWARDS</a>
598
599             <div class="nav-actions">
600                 <a onclick="openModal('loginModal')">SIGN IN</a>
601                 <a onclick="openModal('cartModal')">CART (<span id="cartCount">0</span>)</a>
602             </div>
603         </nav>
604     </header>
```

```
562         /* Responsive Design */
563         @media (max-width: 992px) {
564             #home, #story .story-content {
565                 flex-direction: column;
566                 text-align: center;
567             }
568             .actions {
569                 flex-direction: column;
570                 justify-content: center;
571             }
572             .description { margin: 0 auto 30px auto; }
573             .content-right { margin-top: 60px; }
574             nav { display: none; /* Can be replaced with a hamburger menu later */ }
575             header { justify-content: center; }
576         }
```

Explanation of the problem:

nav-actions are nested inside <nav>, so hiding <nav> also removes the cart and sign-in controls. Below 992px the user can add items but has no way to view or check out the cart. The comment shows the deferral was intentional and then forgotten.

After Correction:



Updated markup:

```
643         <button class="nav-toggle" aria-label="Open navigation menu"
644                 aria-expanded="false" aria-controls="primaryNav"
645                 onclick="toggleNav()">&#9776;</button>
646
647         <nav id="primaryNav">
648             <a href="#menu">MENU</a> <span class="divider">&middot;</span>
649             <a href="#story">OUR STORY</a> <span class="divider">&middot;</span>
650             <a href="#stores">STORES</a>
651         </nav>
652
653         <!-- Moved OUT of <nav> so it is never hidden by the nav breakpoint -->
654         <div class="nav-actions">
655             <button type="button" class="nav-action-btn" onclick="openModal('loginModal')">SIGN IN</button>
656             <button type="button" class="nav-action-btn" onclick="openModal('cartModal')">
657                 CART (<span id="cartCount">0</span>)
658             </button>
659         </div>
```

Updated CSS:

```css
.nav-toggle {
    display: none;                  /* hidden on desktop */
    background: none;
    border: none;
    color: var(--text-main);
    font-size: 1.5rem;
    cursor: pointer;
}

.nav-action-btn {
    background: none;
    border: none;
    color: var(--text-muted);
    font: inherit;
    letter-spacing: 2px;
    font-size: 0.8rem;
    cursor: pointer;
    transition: color 0.3s ease;
}

.nav-action-btn:hover,
.nav-action-btn:focus-visible {
    color: var(--accent-color);
}
```

```css
@media (max-width: 992px) {
    #home, #story .story-content {
        flex-direction: column;
        text-align: center;
    }
    .actions {
        flex-direction: column;
        justify-content: center;
    }
    .description { margin: 0 auto 30px auto; }
    .content-right { margin-top: 60px; }

    .nav-toggle { display: block; order: -1; }

    nav {
        display: none;              /* collapsed by default … */
        position: absolute;
        top: 90px;                  /* directly under the fixed header */
        left: 0;
        width: 100%;
        flex-direction: column;
        gap: 18px;
        padding: 25px 5%;
        background: rgba(62, 38, 27, 0.97);
        border-bottom: 1px solid var(--line-color);
    }

    nav.open { display: flex; }    /* … but reachable via the toggle */
    nav span.divider { display: none; }
```

```
617
618             .nav-actions {
619                 margin-left: 0;
620                 border-left: none;
621                 padding-left: 0;
622             }
623
624             header { justify-content: space-between; }
625         }
```

Updated JavaScript:

```
870     function toggleNav() {
871         const nav = document.getElementById('primaryNav');
872         const toggle = document.querySelector('.nav-toggle');
873         const isOpen = nav.classList.toggle('open');
874         toggle.setAttribute('aria-expanded', isOpen);
875     }
876
877     // Close the mobile menu after a section link is tapped.
878     document.querySelectorAll('#primaryNav a').forEach(link => {
879         link.addEventListener('click', () => {
880             document.getElementById('primaryNav').classList.remove('open');
881             document.querySelector('.nav-toggle').setAttribute('aria-expanded', 'false');
882         });
883     });
```

Explanation of what changed:

The structural fix is the important one: .nav-actions was lifted out of <nav> entirely, so SIGN IN and CART are now siblings of the navigation rather than children of it. They can no longer be switched off by a rule that was only ever meant to collapse the section links. <nav> itself still collapses below 992px, but it is now recoverable through a hamburger button instead of being permanently removed, and header { justify-content: space-between; } keeps the toggle, logo, and cart properly spread out.

I also changed the two <a onclick> controls into real <button type="button"> elements. That was not strictly required by Finding 2, but it partially addresses Finding 7 at zero extra cost buttons are keyboard-focusable and announce themselves correctly to screen readers, which finally makes the :focus styling I had written months ago actually reachable. The toggle carries aria-expanded and aria-controls, so its state is exposed to assistive technology. I left the divider dots hidden in the stacked layout since they only make sense in a horizontal row.

Follow up of Rework #2:

How I know the original problem is solved:

The defining symptom was that no path existed from "Add to Cart" to the cart modal below 992px. I traced that path again at 375px, 768px, 991px, and 1200px. At every width the CART control is visible in the header, the count updates, tapping it opens #cartModal, and Proceed to Checkout is reachable. The structural argument is stronger than the observation, though: the cart control is no longer a descendant of any element that the media query hides, so the specific failure mode cannot recur even if the breakpoint changes.

How I know I did not break something else (regression check):

Behaviour re-checked	**Result**
Desktop (≥993px): nav renders horizontally as before, hamburger hidden	Pass
Desktop: SIGN IN and CART still sit right of the links with the divider border	Pass
991px: hamburger appears, menu opens and closes on tap	Pass
Mobile: tapping MENU scrolls to the section *and* closes the panel	Pass
Cart count still updates while the mobile panel is closed	Pass
Tab key now reaches SIGN IN and CART on desktop, and focus styling is visible	Pass
Login and cart modals still open, close via ×, and close on overlay click	Pass
Opened mobile panel does not overlap the hero headline text	Pass

PART VII — PERSONAL SQA REFLECTION [10 POINTS]

This project was created for our Graphic Design subject. Our professor asked us to develop a website focusing mainly on the front-end part. Because of this, we focused on the design, layout, forms, buttons, and overall user experience instead of developing a complete back-end system. I found the Sign In function the most interesting part of the code to analyze. I tested it many times while building the website, so I expected it to work properly. Because of this, I did not test it deeply. I mostly tested the normal sign-in process and did not try different incorrect passwords or other possible edge cases. I also had comments in my code, such as “Can be replaced with a hamburger menu later,” which stayed in my CSS for a long time.

The duplicated price was also interesting because the price was still being calculated correctly in the cart. I realized that code quality is not only about making sure the code works. It should also be easy to understand, maintain, and modify when changes are needed. I also realized that I sometimes forget tasks when I keep postponing them. This happened when I delayed fixing the lack of a cart on the mobile version of the website. Because of this, I learned that even small tasks should be handled early.

For my next project, I plan to create an SQA checklist to help me track testing and inspection tasks. This will help me avoid forgetting important issues before the project is completed. Overall, this project taught me that good software is not only functional but should also be organized, maintainable, and user-friendly.